

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2025/26)

Lic. em Ciências da Computação
Lic. em Engenharia Informática

Grupo G18

a106807 Marco Rafael Vieira Sèvegrand
a107372 Nuno Henrique Macedo Rebelo
a107326 Guilherme Santos da Costa

Preâmbulo

Em [Cálculo de Programas](#) pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em [Haskell](#) (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em [Haskell](#). Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo [A](#) onde encontrarão as instruções relativas ao *software* a instalar, etc.

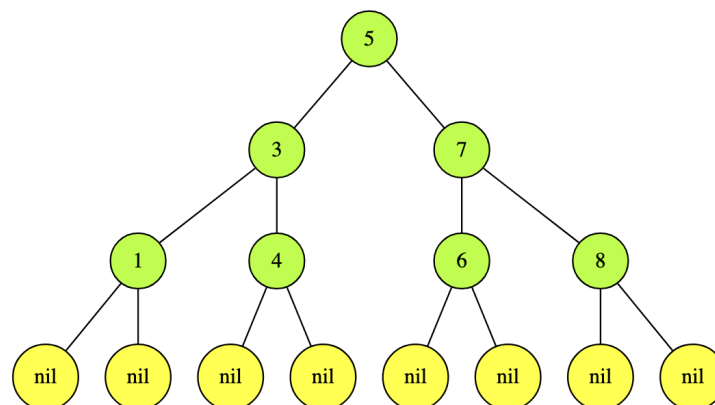
Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Avaliação. Faz parte da avaliação do trabalho a sua defesa por parte dos elementos de cada grupo. Estes devem estar preparados para responder a perguntas sobre *qualquer* dos problemas deste enunciado. A prestação *individual* de cada aluno nessa defesa oral será uma componente importante e diferenciadora da avaliação.

Problema 1

Uma serialização (ou travessia) de uma árvore é uma sua representação sob a forma de uma lista. Na biblioteca *BTree* encontram-se as funções de serialização *inordt*, *preordt* e *postordt*, que fazem as travessias *in-order*, *pre-order* e *post-order*, respectivamente. Todas essas travessias são catamorfismos que percorrem a árvore argumento em regime *depth-first*.

Pretende-se agora uma função *bfordr* que faça a travessia em regime *breadth-first*, isto é, por níveis. Por exemplo, para a árvore t_1 dada em anexo e mostrada na figura a seguir,



a função deverá dar a lista

[5, 3, 7, 1, 4, 6, 8]

em que se vê como os níveis 5, depois 3, 7 e finalmente 1, 4, 6, 8 foram percorridos.

Pretendemos propor duas versões dessa função:

1. Uma delas envolve um catamorfismo de *BTrees*:

$$\begin{aligned} \text{bfsLevels} &:: \text{BTree } a \rightarrow [a] \\ \text{bfsLevels} &= \text{concat} \cdot \text{levels} \end{aligned}$$

Complete a definição desse catamorfismo:

$$\begin{aligned} \text{levels} &:: \text{BTree } a \rightarrow [[a]] \\ \text{levels} &= \llbracket g\text{levels} \rrbracket \end{aligned}$$

2. A segunda proposta,

$$\text{bft} :: \text{BTree } a \rightarrow [a]$$

deverá basear-se num anamorfismo de listas.

Sugestão: estudar o artigo [?] cujo PDF está incluído no material deste trabalho. Quando fizer testes ao seu código pode, se desejar, usar funções disponíveis na biblioteca *Exp* para visualizar as árvores em GraphViz (formato .dot).

Justifique devidamente a sua resolução, que deverá vir acompanhada de diagramas explicativos. Como já se disse, valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 2

Considere a seguinte função em Haskell:

```
f x = wrapper · worker where
  wrapper = head
  worker 0 = start x
  worker (n + 1) = loop x (worker n)
  loop x [s, h, k, j, m] =
    [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
  start x = [x, x ↑ 3, 6, 20, 22]
```

Pode-se provar pela lei de recursividade mútua que $f\ x\ n$ calcula o seno hiperbólico de x , $\sinh x$, para n aproximações da sua série de Taylor. Faça a derivação da função dada a partir da referida série de Taylor, apresentando todos os cálculos justificativos, tal como se faz para outras funções no capítulo respectivo do texto base desta UC [?].

Problema 3

Quem em Braga observar, ao fim da tarde, o tráfego onde a Avenida Clairmont Fernand se junta à N101, aproximadamente na coordenada [41°33'46.8"N 8°24'32.4"W](#) — ver as setas da figura que se segue — reparará nas sequências imparáveis (infinitas!) de veículos provenientes dessas vias de circulação.

Mas também irá observar um comportamento interessante por parte dos condutores desses veículos: por regra, *cada carro numa via deixa passar, à sua frente, exactamente outro carro da outra via*.



Este comportamento *civilizado* chama-se *fair-merge* (ou *fair-interleaving*) de duas sequências infinitas, também designadas *streams* em ciência da computação. Seja dado o tipo dessas sequências em Haskell,

data *Stream* *a* = *Cons* (*a*, *Stream* *a*) **deriving** *Show*

para o qual se define também:

out (*Cons* (*x*, *xs*)) = (*x*, *xs*)

O referido comportamento civilizado pode definir-se, em Haskell, da forma seguinte:¹

```
fair_merge :: (Stream a, Stream a) + (Stream a, Stream a) → Stream a
fair_merge = [h, k] where
  h (Cons (x, xs), y) = Cons (x, k (xs, y))
  k (x, Cons (y, ys)) = Cons (y, h (x, ys))
```

Defina *fair_merge* como um **anamorfismo** de *Streams*, usando o combinador

$\llbracket g \rrbracket = \text{Cons} \cdot (\text{id} \times \llbracket g \rrbracket) \cdot g$

e a seguinte estratégia:

- Derivar a lei **dual** da recursividade mútua,

$$[f, g] = \llbracket [h, k] \rrbracket \equiv \begin{cases} \text{out} \cdot f = F [f, g] \cdot h \\ \text{out} \cdot g = F [f, g] \cdot k \end{cases} \quad (1)$$

tal como se fez, nas aulas, para a que está no formulário.

- Usar (1) na resolução do problema proposto.

Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Problema 4

Como se sabe, é possível pensarmos em catamorfismos, anamorfismos etc *probabilísticos*, quer dizer, programas recursivos que dão distribuições como resultados. Por exemplo, podemos pensar num combinador

pcataList :: (() + (*a*, *b*) → *Dist* *b*) → [*a*] → *Dist* *b*

¹ O facto das sequências serem infinitas não nos deve preocupar, pois em Haskell isso é lidado de forma transparente por [lazy evaluation](#).

que é muito parecido com

$$(\cdot) :: () \rightarrow (a, b) \rightarrow b \rightarrow [a] \rightarrow b$$

da biblioteca [List](#). A principal diferença é que o gene de *pcataList* é uma função probabilística.

Como exemplo de utilização, recorde-se que ([zero, add]) soma todos os elementos da lista argumento, por exemplo:

$$(\text{[zero, add]}) [20, 10, 5] = 35.$$

Considere-se agora a função *padd* (adição probabilística) que, com probabilidade 90% soma dois números e com probabilidade 10% os subtrai:

$$\text{padd } (a, b) = D [(a + b, 0.9), (a - b, 0.1)]$$

Se se correr

$$d4 = \text{pcataList } [\text{pzero, padd}] [20, 10, 5] \text{ where } \text{pzero} = \text{return} \cdot \text{zero}$$

obter-se-á:

```
35  81.0%
25   9.0%
 5   9.0%
15   1.0%
```

Com base neste exemplo, resolva o seguinte

Problema: Uma unidade militar pretende enviar uma mensagem urgente a outra, mas tem o aparelho de telegrafia meio avariado. Por experiência, o telegrafista sabe que a probabilidade de uma palavra se perder (não ser transmitida) é 5%; e que, no final de cada mensagem, o aparelho envia o código "stop", mas (por estar meio avariado), falha 10% das vezes.

Qual a probabilidade de a palavra "atacar" da mensagem

`words "Vamos atacar hoje"`

se perder, isto é, o resultado da transmissão ser ["Vamos", "hoje", "stop"]? E a de seguirem todas as palavras, mas faltar o "stop" no fim? E a da transmissão ser perfeita?

Responda a estas perguntas encontrando *gene* tal que

`transmitir = pcataList gene`

descreve o comportamento do aparelho. Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na [página da disciplina](#) na internet.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

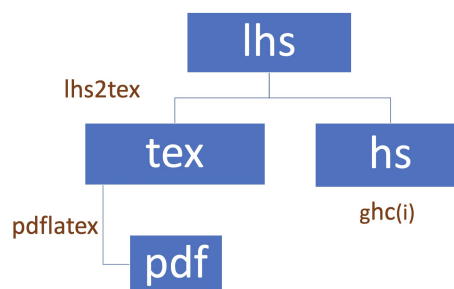
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita “**literária**” [?], cujo princípio base é o seguinte:

Um programa e a sua documentação devem coincidir.

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2526t.pdf` que está a ler é já um exemplo de **programação literária**: foi gerado a partir do texto fonte `cp2526t.lhs`¹ que encontrará no **material pedagógico** desta disciplina descompactando o ficheiro `cp2526t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código **Haskell** que ele inclui:



Vê-se assim que, para além do **GHCI**, serão necessários os executáveis **pdflatex** e **lhs2TeX**. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do **Docker** tal como a seguir se descreve.

B Docker

Recomenda-se o uso do **container** cuja imagem é gerada pelo **Docker** a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2526t.zip`. Este **container** deverá ser usado na execução do **GHCI** e dos comandos relativos ao **LaTeX**. (Ver também a `Makefile` que é disponibilizada.)

Após **instalar o Docker** e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2526t .  
$ docker run -v ${PWD}:/cp2526t -it cp2526t
```

NB: O objetivo é que o container seja usado *apenas* para executar o **GHCI** e os comandos relativos ao **LaTeX**. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2526t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2526t` no **container** sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no **container**, executando:

¹ O sufixo ‘lhs’ quer dizer *literate Haskell*.

```
$ lhs2TeX cp2526t.lhs > cp2526t.tex
$ pdflatex cp2526t
```

[lhs2TeX](#) é o pre-processador que faz “pretty printing” de código Haskell em [L^AT_EX](#) e que faz parte já do [container](#). Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2526t.lhs` é executável e contém o “kit” básico, escrito em [Haskell](#), para realizar o trabalho. Basta executar

```
$ ghci cp2526t.lhs
```

Abra o ficheiro `cp2526t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo [GHCi](#) para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo [G](#) com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com [BibT_EX](#)) e o índice remissivo (com [makeindex](#)),

```
$ bibtex cp2526t.aux
$ makeindex cp2526t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no [container](#).)

No anexo [F](#) disponibiliza-se algum código [Haskell](#) relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da [programação literária](#) para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo [D](#) que se segue.

D Como exprimir cálculos e diagramas em L^AT_EX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte ([lhs](#)) do que está a ler¹ onde se obtém o efeito seguinte:²

$$\begin{aligned} id &= \langle f, g \rangle \\ \equiv \quad &\{ \text{universal property} \} \end{aligned}$$

¹ Procure e.g. por "sec:diagramas".

² Exemplos tirados de [?].

$$\begin{aligned}
& \begin{cases} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{cases} \\
\equiv & \quad \{ \text{identity} \} \\
& \begin{cases} \pi_1 = f \\ \pi_2 = g \end{cases} \\
& \square
\end{aligned}$$

Os diagramas podem ser produzidos recorrendo à *package* [xymatrix](#), por exemplo:

$$\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
\downarrow \langle g \rangle & & \downarrow id + \langle g \rangle \\
B & \xleftarrow{g} & 1 + B
\end{array}$$

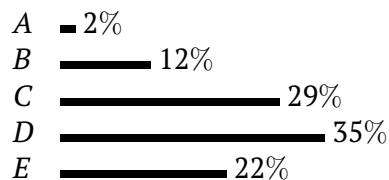
E O mónade das distribuições probabilísticas

Mónades são funtores com propriedades adicionais que nos permitem obter efeitos especiais em programação. Por exemplo, a biblioteca [Probability](#) oferece um mónade para abordar problemas de probabilidades. Nesta biblioteca, o conceito de distribuição estatística é captado pelo tipo

$$\text{newtype Dist } a = D \{ \text{unD} :: [(a, \text{ProbRep})] \} \quad (2)$$

em que *ProbRep* é um real de 0 a 1, equivalente a uma escala de 0 a 100%.

Cada par (a, p) numa distribuição $d :: \text{Dist } a$ indica que a probabilidade de a é p , devendo ser garantida a propriedade de que todas as probabilidades de d somam 100%. Por exemplo, a seguinte distribuição de classificações por escalões de A a E ,



será representada pela distribuição

$$\begin{aligned}
d1 &:: \text{Dist Char} \\
d1 &= D [('A', 0.02), ('B', 0.12), ('C', 0.29), ('D', 0.35), ('E', 0.22)]
\end{aligned}$$

que o [GHCi](#) mostrará assim:

```

'D'  35.0%
'C'  29.0%
'E'  22.0%
'B'  12.0%
'A'   2.0%

```

É possível definir geradores de distribuições, por exemplo distribuições *uniformes*,

$$d2 = \text{uniform } (\text{words "Uma frase de cinco palavras"})$$

isto é


```

    "Uma"    20.0%
    "cinco"  20.0%
    "de"     20.0%
    "frase"  20.0%
    "palavras" 20.0%

```

distribuição *normais*, eg.

```
d3 = normal [10..20]
```

etc.¹ Dist forma um **mónade** cuja unidade é $\text{return } a = D [(a, 1)]$ e cuja composição de Kleisli é (simplificando a notação)

$$(f \bullet g) a = [(y, q * p) \mid (x, p) \leftarrow g a, (y, q) \leftarrow f x]$$

em que $g : A \rightarrow \text{Dist } B$ e $f : B \rightarrow \text{Dist } C$ são funções **monádicas** que representam *computações probabilísticas*.

Este mónade é adequado à resolução de problemas de *probabilidades e estatística* usando programação funcional, de forma elegante e como caso particular da programação monádica.

F Código fornecido

Problema 1

Árvores exemplo:

```

t1 :: BTree Int
t1 = Node (5, (Node (3, (Node (1, (Empty, Empty)), Node (4, (Empty, Empty)))),
    Node (7, (Node (6, (Empty, Empty)), Node (8, (Empty, Empty)))))
t2 :: BTree Int
t2 =
    node 1
        (node 2 (node 4 Empty Empty) (node 5 Empty Empty))
        (node 3 (node 6 Empty Empty) (node 7 Empty Empty))
t3 :: BTree Char
t3 =
    node 'A'
        (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
        (node 'E' Empty Empty)
t4 :: BTree Char
t4 =
    node 'A'
        (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
        Empty
t5 :: BTree Int
t5 =
    node 1

```

¹ Para mais detalhes ver o código fonte de [Probability](#), que é uma adaptação da biblioteca [PFP](#) ("Probabilistic Functional Programming"). Para quem quiser saber mais recomenda-se a leitura do artigo [?].

$(node\ 2\ (node\ 4\ Empty\ Empty)\ Empty)$
 $(node\ 3\ Empty\ (node\ 5\ (node\ 6\ Empty\ Empty)\ Empty))$
 $node\ a\ b\ c = Node\ (a,\ (b,\ c))$

G Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o “layout” que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

Uma travessia em largura (breadth-first) de uma árvore processa os nós nível por nível, começando pela raiz. Existem diferentes formas de implementar esta travessia, cada uma explorando um combinador recursivo distinto.

Primeira abordagem: catamorfismo de árvores

A função *levels* agrupa elementos por profundidade, retornando uma lista de listas onde cada sub-lista representa um nível. Depois, *bfsLevels* concatena simplesmente estes níveis numa única lista, obtendo o resultado breadth-first.

Embora o resultado final seja breadth-first, o catamorfismo processa internamente a árvore de baixo para cima (visitando primeiro as subárvores). Quando chegamos a um nó, temos já processadas as subárvores esquerda e direita, recebendo listas de níveis para cada uma. O nó actual forma um novo nível no topo, e os restantes níveis são fundidos par a par através de *zipLevels*, alinhando elementos da mesma profundidade das duas subárvores.

Implementação

```

glevels :: () + (a, ([[a]], [[a]])) → [[a]]
glevels (i1 ()) = []
glevels (i2 (a, (ls, rs))) = [a] : zipLevels ls rs
where
  zipLevels [] rs = rs
  zipLevels ls [] = ls
  zipLevels (l : ls) (r : rs) = (l ++ r) : zipLevels ls rs

```

Diagrama do catamorfismo

Catamorfismo *levels*:

$$\begin{array}{ccc}
 BTree\ a & \xleftarrow{in_{BTree}} & 1 + a \times (BTree\ a \times BTree\ a) \\
 \downarrow levels & & \downarrow id + id \times (levels \times levels) \\
 [[a]] & \xleftarrow{glevels} & 1 + a \times ([[a]] \times [[a]])
 \end{array}$$

O diagrama mostra como *levels* decompõe a árvore (via in_{BTree}), aplica *levels* recursivamente às subárvores, e depois combina o resultado usando *glevels*. O gene *glevels* trata dois casos: árvore vazia

(retorna lista vazia) e nó com valor a e subárvores processadas (cria um novo nível com a e funde os restantes).

Segunda abordagem: anamorfismo de listas

A função *bft* implementa uma travessia breadth-first através de um anamorfismo que utiliza uma fila de árvores. Ao contrário do catamorfismo que destrói a estrutura, o anamorfismo constrói incrementalmente a lista resultado.

O algoritmo funciona da seguinte forma: iniciamos com uma fila contendo apenas a raiz da árvore. Depois, repetidamente, removemos a árvore da frente da fila. Se for *Empty*, ignoramo-la e prosseguimos. Se for um nó *Node*, extraímos o seu valor (que é emitido para a lista resultado) e adicionamos os seus filhos no fim da fila. Este regime FIFO (first-in-first-out) garante que visitamos os nós exactamente na ordem breadth-first: primeiro todos os nós do nível 0 (raiz), depois nível 1, depois nível 2, e assim sucessivamente.

Implementação

```

bft t = [(gbf)] [t]
where
  gbf [] = i1 ()
  gbf (Empty : ts) = gbf ts
  gbf (Node (a, (l, r)) : ts) = i2 (a, ts ++ [l, r])

```

Diagrama do anamorfismo

Anamorfismo *bft*:

$$\begin{array}{ccc}
 [BTree\ a] & \xrightarrow{gbf} & 1 + a \times [BTree\ a] \\
 bft \downarrow & & \downarrow id + id \times [(gbf)] \\
 [a] & \xleftarrow{inList} & 1 + a \times [a]
 \end{array}$$

O gene *gbf* define o passo de construção da lista. A fila de árvores é consumida por três casos: se está vazia, o processo termina ($i_1 ()$); se contém uma árvore vazia, ignoramo-la recursivamente; se contém um nó, emitimos o seu valor e continuamos com os restantes elementos da fila mais os seus filhos. A operação $ts ++ [l, r]$ coloca os filhos no fim, não no início, mantendo a ordem breadth-first.

Problema 2

O objectivo deste problema é derivar a implementação da função seno hiperbólico ($\sinh x$) a partir da sua série de Taylor, utilizando a lei da recursividade mútua e catamorfismos de números naturais.

A série de Taylor para $\sinh x$

$$\sinh x = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!} = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \dots$$

Para calcular esta série de forma iterativa e eficiente, mantemos um estado que evolui em cada iteração, e usamos um catamorfismo de números naturais para repetir este processo n vezes.

Estrutura do estado iterativo

O estado é representado por um vector de cinco componentes $[s, h, k, j, m]$:

- s : Soma acumulada dos termos da série.
- h : Numerador do termo atual (x^{2n+1}).
- k : Denominador do termo atual $((2n + 1)!)$.
- j : Produto dos dois próximos factores da sequência de denominadores.
- m : Incremento linear para j .

A transição de um estado para o seguinte é dada pela função *loop*:

$$\text{novo estado} = \left[\frac{h}{k} + s, x^2 * h, k * j, j + m, m + 8 \right]$$

Estado inicial para $n = 0$:

$$\text{start } x = [x, x^3, 6, 20, 22]$$

Este estado inicial contém já o primeiro termo da série ($s = x$) e prepara o cálculo do segundo termo (com $h = x \uparrow 3$ e $k = 6 = 3!$).

Exemplo de iterações para $x = 2$

A tabela seguinte mostra a evolução do estado ao longo das primeiras iterações:

n	s	h	k	j	m
0	2.000000	8	6	20	22
1	3.333333	32	120	42	30
2	3.600000	128	5040	72	38
3	3.625397	512	362880	110	46
4	3.626808	2048	39916800	156	54
5	3.626859	8192	6.23×10^9	210	62

Observa-se que:

- O valor de s converge rapidamente para $\sinh(2) \approx 3.626860$.
- O numerador h cresce como potências de $x^2 = 4$: $8 \rightarrow 32 \rightarrow 128 \rightarrow 512 \rightarrow \dots$ (correspondendo a $2^3, 2^5, 2^7, \dots$).
- O denominador k cresce factorialmente: $6 \rightarrow 120 \rightarrow 5040 \rightarrow \dots$ (correspondendo a $3!, 5!, 7!, \dots$).
- Os valores j e m seguem progressões que garantem a multiplicação correta dos factoriais sucessivos.

Implementação

```
f_sinh :: Double → Int → Double
f_sinh x n = head (worker n)
  where
    worker = (gene)
    gene = [start x, loop x]
    loop x [s, h, k, j, m] = [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
    start x = [x, x ↑ 3, 6, 20, 22]
```

Diagrama do catamorfismo

$$\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{in_{Nat}} & 1 + \mathbb{N}_0 \\
\downarrow worker & & \downarrow id+worker \\
[Double] & \xleftarrow{gene} & 1 + [Double]
\end{array}$$

O catamorfismo *worker* aplica iterativamente a função *loop* x começando no estado inicial *start* x . A função *gene* trata dois casos: para o caso base (zero), retorna o estado inicial; para o caso indutivo, aplica *loop* x ao estado resultante da iteração anterior. Por fim, *head* extrai o primeiro componente do vector final, que é a soma acumulada s .

Problema 3

O problema do fair-merge consiste em intercalar duas sequências infinitas (streams) de forma alternada: um elemento da primeira, um da segunda, um da primeira, etc. O objectivo é expressar a função *fair_merge* como um anamorfismo de streams, derivando o *gene* a partir da lei dual da recursividade mútua.

Ponto de partida: a definição dada

A função *fair_merge* é definida por duas funções mutuamente recursivas:

$$\begin{aligned}
h(Cons(x, xs), y) &= Cons(x, k(xs, y)) \\
k(x, Cons(y, ys)) &= Cons(y, h(x, ys))
\end{aligned}$$

onde h extrai do primeiro stream e k extrai do segundo, alternando entre si.

A função final é *fair_merge* = $[h, k]$, que escolhe entre começar por h ou k conforme o argumento seja i_1 ou i_2 .

Derivação da lei dual da recursividade mútua

Queremos encontrar um *gene* g tal que $[h, k] = \llbracket g \rrbracket$. Partindo da propriedade universal dos anamorfismos:

$$\begin{aligned}
f &= \llbracket g \rrbracket \\
&\equiv \{ \text{universal-ana (56)} \} \\
out \cdot f &= (id \times f) \cdot g
\end{aligned}$$

Aplicando a $f = [h, k]$ e $g = [gh, gk]$:

$$\begin{aligned}
[h, k] &= \llbracket [gh, gk] \rrbracket \\
&\equiv \{ \text{universal-ana (56)} \} \\
out \cdot [h, k] &= (id \times [h, k]) \cdot [gh, gk] \\
&\equiv \{ \text{fusão-+ (21)} \} \\
[out \cdot h, out \cdot k] &= ((id \times [h, k]) \cdot gh, (id \times [h, k]) \cdot gk) \\
&\equiv \{ \text{eq-+ (28)} \} \\
&\equiv \begin{cases} out \cdot h = (id \times [h, k]) \cdot gh \\ out \cdot k = (id \times [h, k]) \cdot gk \end{cases}
\end{aligned}$$

□

A lei dual da recursividade mútua permitiu derivar os genes gh e gk a partir das funções mutuamente recursivas h e k .

Aplicação ao fair-merge

Calculemos $\text{out} \cdot h$ e $\text{out} \cdot k$:

$$\begin{aligned}
& \text{out} \cdot h (\text{Cons } (x, xs), y) \\
= & \quad \{ \text{definição de } h \} \\
& \text{out } (\text{Cons } (x, k (xs, y))) \\
= & \quad \{ \text{definição de out } \} \\
& (x, k (xs, y)) \\
= & \quad \{ \text{cancelamento-+ (19)} \} \\
& (x, [h, k] (i_2 (xs, y))) \\
= & \quad \{ \text{def-}\times \text{ (79)} \} \\
& (id \times [h, k]) (x, i_2 (xs, y)) \\
& \square
\end{aligned}$$

Portanto, $gh (\text{Cons } (x, xs), y) = (x, i_2 (xs, y))$.

Analogamente para k :

$$\begin{aligned}
& \text{out} \cdot k (x, \text{Cons } (y, ys)) \\
= & \quad \{ \text{definição de } k \} \\
& \text{out } (\text{Cons } (y, h (x, ys))) \\
= & \quad \{ \text{definição de out } \} \\
& (y, h (x, ys)) \\
= & \quad \{ \text{cancelamento-+ (19)} \} \\
& (y, [h, k] (i_1 (x, ys))) \\
= & \quad \{ \text{def-}\times \text{ (79)} \} \\
& (id \times [h, k]) (y, i_1 (x, ys)) \\
& \square
\end{aligned}$$

Portanto, $gk (x, \text{Cons } (y, ys)) = (y, i_1 (x, ys))$.

O gene do anamorfismo

Combinando gh e gk num único gene $gene = [gh, gk]$:

$$\begin{aligned}
gene (i_1 (\text{Cons } (x, xs), y)) &= (x, i_2 (xs, y)) \\
gene (i_2 (x, \text{Cons } (y, ys))) &= (y, i_1 (x, ys))
\end{aligned}$$

O gene extrai um elemento do stream apropriado e *troca o lado do Either* para garantir a alternância no passo seguinte.

Implementação

$$\begin{aligned}
\text{fair_merge}' &:: (\text{Stream } a, \text{Stream } a) + (\text{Stream } a, \text{Stream } a) \rightarrow \text{Stream } a \\
\text{fair_merge}' &= \llbracket gene \rrbracket
\end{aligned}$$

where

$gene\ (i_1\ (Cons\ (x, xs), y)) = (x, i_2\ (xs, y))$
 $gene\ (i_2\ (x, Cons\ (y, ys))) = (y, i_1\ (x, ys))$

Diagrama do anamorfismo

$$\begin{array}{ccc}
 (S, S) + (S, S) & \xrightarrow{gene} & a \times ((S, S) + (S, S)) \\
 \downarrow fair_merge' & & \downarrow id \times fair_merge' \\
 S & \xleftarrow{Cons} & a \times S
 \end{array}$$

onde $S = Stream\ a$

O anamorfismo constrói a stream resultado aplicando repetidamente o `gene`: cada aplicação extrai um elemento e produz um novo estado com o `Either` invertido, garantindo a alternância perfeita entre os dois streams de entrada.

Verificação da solução

Para testar a implementação, definimos as seguintes funções auxiliares:

```
-- Função para visualizar os primeiros n elementos de um stream
takeStream :: Int -> Stream a -> [a]
takeStream 0 _ = []
takeStream n (Cons (x, xs)) = x : takeStream (n - 1) xs

-- Gerar stream de naturais começando em n, incrementando de step
natsFrom :: Int -> Int -> Stream Int
natsFrom start step = [(λx -> (x, x + step))] start

-- Streams de teste
nats1 :: Stream Int
nats1 = natsFrom 1 1      -- 1, 2, 3, 4, 5, ...
nats10 :: Stream Int
nats10 = natsFrom 10 10  -- 10, 20, 30, 40, ...

-- Testes pré-definidos
test1 :: [Int]
test1 = takeStream 16 (fair_merge' (i1 (nats1, nats10)))
test2 :: [Int]
test2 = takeStream 16 (fair_merge' (i2 (nats1, nats10)))
```

O resultado de `test1` mostra a alternância começando pelo stream A (1, 2, 3, ...), enquanto `test2` mostra a alternância começando pelo stream B (10, 20, 30, ...). Ambos respeitam perfeitamente o padrão de `fair-merge`: um elemento de cada stream alternadamente.

Problema 4

O problema consiste em modelar a transmissão de uma mensagem através de um aparelho de telegrafia com falhas probabilísticas, usando um catamorfismo probabilístico.

Especificação do problema

A mensagem a transmitir é `words "Vamos atacar hoje"` = ["Vamos", "atacar", "hoje"]. O aparelho tem o seguinte comportamento:

- Cada palavra tem 5% de probabilidade de se perder (95% de sucesso)
- No final, o aparelho envia "stop", mas falha 10% das vezes (90% de sucesso)

Queremos encontrar *gene* tal que $transmitir = pcataList\ gene$ descreva este comportamento.

Análise do catamorfismo probabilístico

O combinador *pcataList* é a versão probabilística de $(\cdot)$:

$$\begin{aligned} pcataList\ g &= h \\ \textbf{where} \\ h\ [] &= g\ (i_1\ ()) \\ h\ (x : xs) &= h\ xs \gg \lambda r \rightarrow g\ (i_2\ (x, r)) \end{aligned}$$

A diferença fundamental é que o gene *g* retorna uma distribuição *Dist b*, e a composição usa o operador monádico $\gg$ para propagar as probabilidades.

A recursão processa a lista da direita para a esquerda:

$$h\ [a_1, a_2, a_3] = h\ [a_2, a_3] \gg (\lambda r \rightarrow g\ (\text{Right}\ (a_1, r)))$$

Isto significa que *r* representa o resultado (já probabilístico) da transmissão da cauda da lista, e o gene decide se adiciona ou não o elemento actual.

Derivação do gene

O gene tem tipo $() + (String, [String]) \rightarrow \text{Dist}\ [String]$ e deve tratar dois casos:

Caso base – *gene* $(i_1\ ())$:

Não há mais palavras. O aparelho tenta enviar "stop":

- Com probabilidade 0.9: sucesso, retorna ["stop"]
- Com probabilidade 0.1: falha, retorna []

Caso recursivo – *gene* $(i_2\ (palavra, resto))$:

Recebe a palavra actual e *resto*, que é o resultado da transmissão da cauda. O aparelho tenta enviar a palavra:

- Com probabilidade 0.95: sucesso, retorna *palavra : resto*
- Com probabilidade 0.05: falha, retorna *resto* (palavra perdida)

Implementação

$$\begin{aligned} gene &:: () + (String, [String]) \rightarrow \text{Dist}\ [String] \\ gene\ (i_1\ ()) &= \text{choose}\ 0.9\ ["stop"]\ [] \\ gene\ (i_2\ (palavra, resto)) &= \text{choose}\ 0.95\ (palavra : resto)\ resto \end{aligned}$$

Diagrama do catamorfismo probabilístico

$$\begin{array}{ccc} [String] & \xleftarrow{in_{List}} & 1 + String \times [String] \\ pcataList\ gene \downarrow & & \downarrow id + id \times pcataList\ gene \\ \text{Dist}\ [String] & \xleftarrow{gene'} & 1 + String \times \text{Dist}\ [String] \end{array}$$

onde $gene' = [gene \cdot i_1, \lambda(a, d) \rightarrow d \gg \lambda r \rightarrow gene (i_2 (a, r))]$ combina o gene com a propagação monádica das probabilidades.

Verificação: cálculo manual das probabilidades

Seja $msg = ["Vamos", "atacar", "hoje"]$. Calculemos $pcataList\ gene\ msg$:

Passo 1 – $h [] = gene (i_1 ()) = choose\ 0.9\ ["stop"] []$

Distribuição inicial:

$$D_0 = \begin{cases} ["stop"] & \text{com prob. 0.9} \\ [] & \text{com prob. 0.1} \end{cases}$$

Passo 2 – $h ["hoje"] = h [] \gg \lambda r \rightarrow gene (i_2 ("hoje", r))$

Para cada resultado r de D_0 , aplicamos $gene (i_2 ("hoje", r))$:

- $r = ["stop"]$: $choose\ 0.95\ ["hoje", "stop"] ["stop"]$
- $r = []$: $choose\ 0.95\ ["hoje"] []$

Distribuição após "hoje":

$$D_1 = \begin{cases} ["hoje", "stop"] & 0.9 \times 0.95 = 0.855 \\ ["stop"] & 0.9 \times 0.05 = 0.045 \\ ["hoje"] & 0.1 \times 0.95 = 0.095 \\ [] & 0.1 \times 0.05 = 0.005 \end{cases}$$

Passo 3 – $h ["atacar", "hoje"]$: análogo, multiplica cada linha por 0.95 (sucesso) ou 0.05 (falha).

Passo 4 – $h ["Vamos", "atacar", "hoje"]$: idem.

Resultados finais

Após propagar as probabilidades, obtemos:

1. **Transmissão perfeita**: $["Vamos", "atacar", "hoje", "stop"]$

$$P = 0.95 \times 0.95 \times 0.95 \times 0.90 = 0.95^3 \times 0.90 \approx 0.7716 \quad (77.16\%)$$

2. **Falta o "stop"**: $["Vamos", "atacar", "hoje"]$

$$P = 0.95 \times 0.95 \times 0.95 \times 0.10 = 0.95^3 \times 0.10 \approx 0.0857 \quad (8.57\%)$$

3. **Perda de "atacar"**: $["Vamos", "hoje", "stop"]$

$$P = 0.95 \times 0.05 \times 0.95 \times 0.90 \approx 0.0406 \quad (4.06\%)$$

Testes em Haskell

```
transmissao :: Dist [String]
transmissao = transmitir (words "Vamos atacar hoje")
-- Probabilidade de transmissão perfeita
probPerfeita :: Probability
probPerfeita = (≡ ["Vamos", "atacar", "hoje", "stop"]) ?? transmissao
-- Probabilidade de faltar o "stop"
probSemStop :: Probability
```

```

probSemStop = (≡ ["Vamos", "atacar", "hoje"]) ?? transmissao
  -- Probabilidade de perder "atacar"
probPerdaAtacar :: Probability
probPerdaAtacar = (≡ ["Vamos", "hoje", "stop"]) ?? transmissao

```

Resultados esperados:

- $probPerfeita \approx 0.7716$
- $probSemStop \approx 0.0857$
- $probPerdaAtacar \approx 0.0406$